GPUs: From Graphics to AI

Part 2: Programming the GPU

CUDA C++ and the libraries above it

Hardware Description Languages

Review from Part 1

CPU: a few big cores, low **latency**

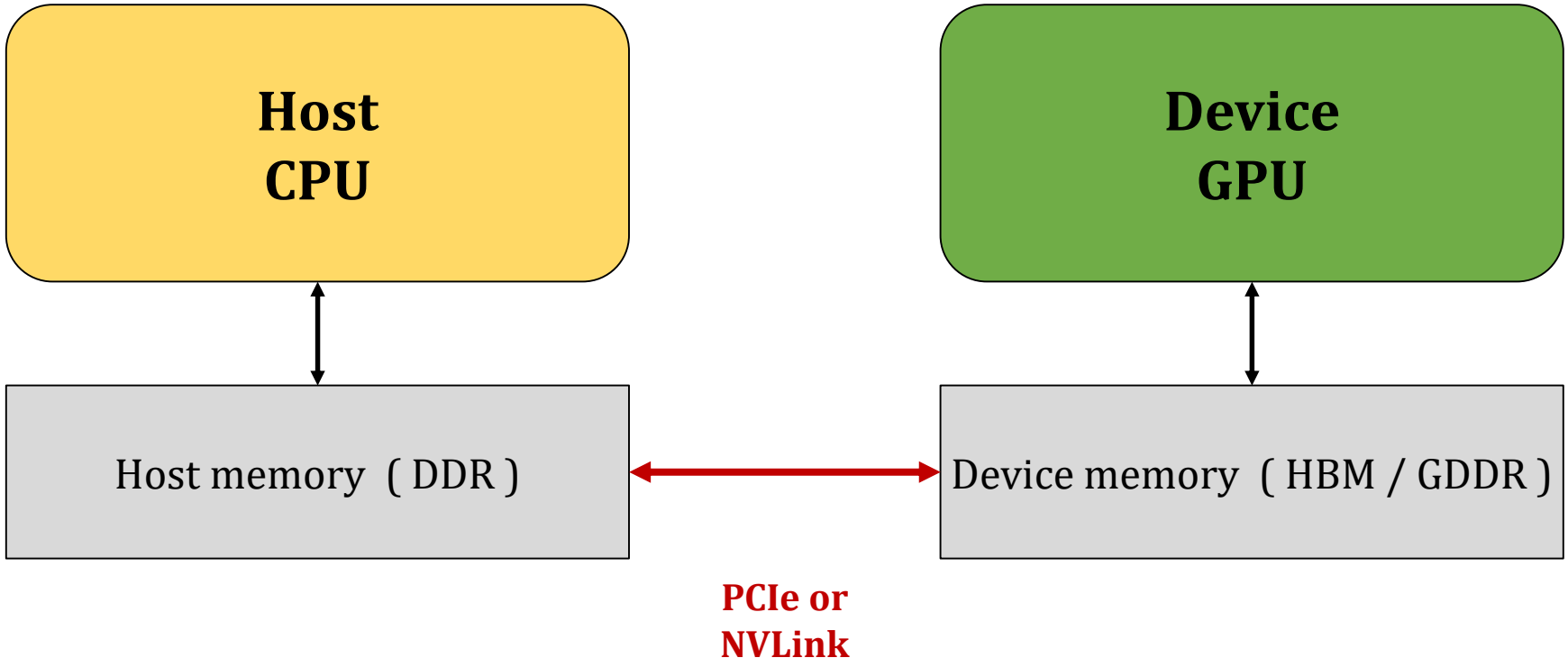
GPU: thousands of small lanes, high **throughput**

SIMD: one instruction drives many lanes

The GPU hides memory latency by

switching between groups of threads

Host and Device



Two processors, **two separate memories**

Data must be copied across - and that copy is slow

Every CUDA Program

1. Allocate memory on the device
2. Copy inputs host → device
3. **Launch a kernel** on many threads
4. Copy results device → host
5. Free device memory

What Is a Kernel?

A function that runs **on the GPU**

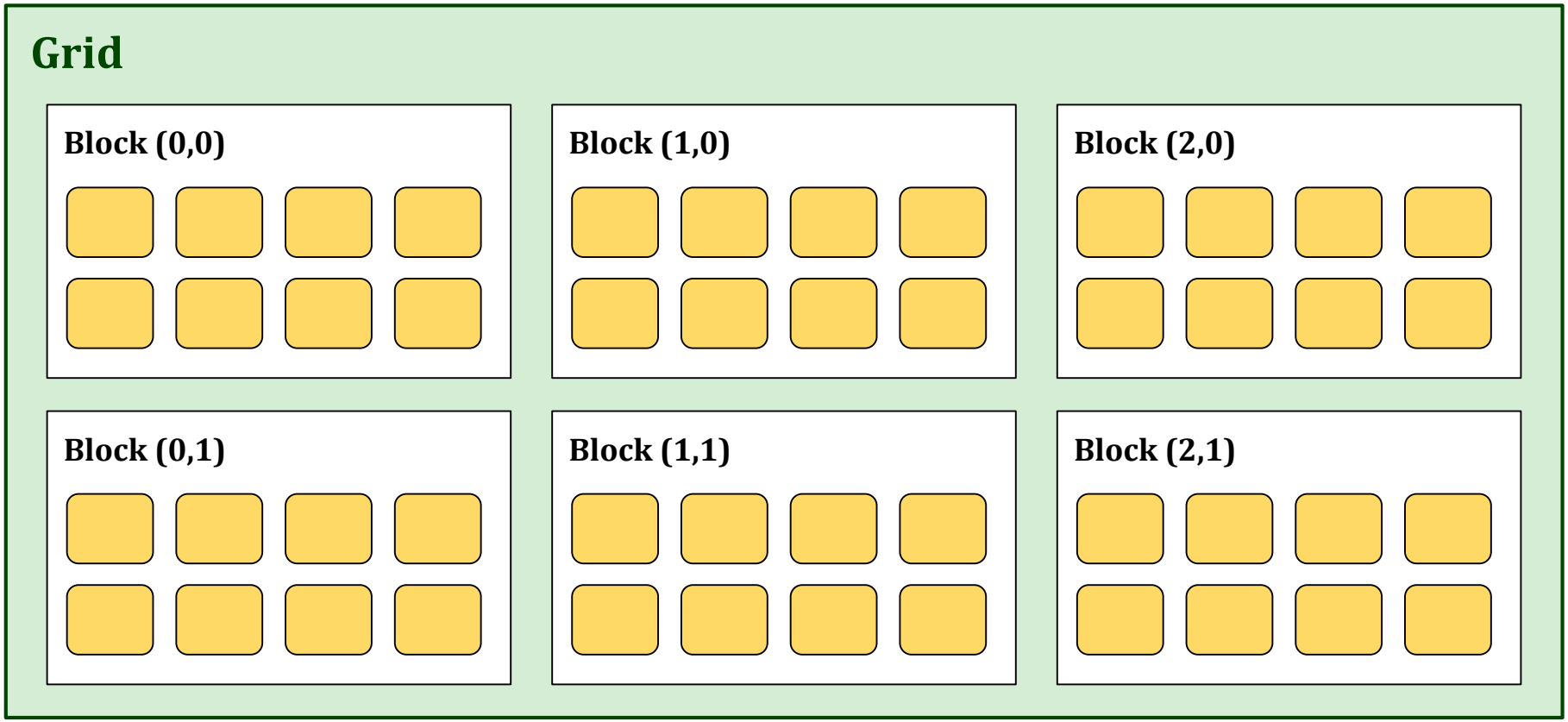
Marked with `__global__`

called from the host, runs on the device

Written for **one** thread

The hardware runs it on **thousands** at once

Grid, Block, Thread



A **grid** of **blocks**, each block a group of **threads**

Threads in one block can cooperate; blocks are independent

Who Am I?

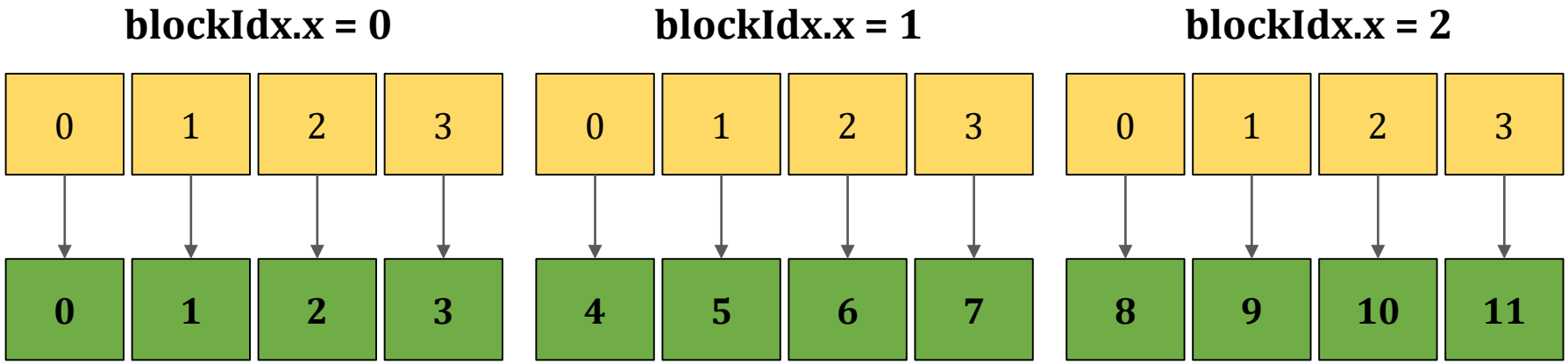
Every thread reads built-in variables:

`threadIdx`	my index inside the block
`blockIdx`	my block's index in the grid
`blockDim`	threads per block
`gridDim`	blocks in the grid

Each has `.x .y .z` for 1-D, 2-D, 3-D problems

From Block and Thread to i

```
int i = blockIdx.x * blockDim.x + threadIdx.x;
```



amber = threadIdx.x *green = global index i*

With blockDim.x = 4: block 2, thread 1 → i = 2 · 4 + 1 = **9**

Every thread gets a **unique** i

Vector Add on the CPU

```
void vec_add(const float *a, const float *b,
                  float *c, int n)
{
    for (int i = 0; i < n; i++)
        c[i] = a[i] + b[i];
}
```

One thread does all n additions, one after another

Vector Add on the GPU

```
__global__
void vec_add(const float *a, const float *b,
                  float *c, int n)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    if (i < n)
        c[i] = a[i] + b[i];
}
```

The loop is gone: each thread does one addition

Vortex Example:Relu

```
include <vx_spawn2.h> // rectified linear unit RELU
#include "common.h"

__kernel void kernel_main(kernel_arg_t* __UNIFORM__ arg) {
        auto src0_ptr = reinterpret_cast<TYPE*>(arg->src0_addr);
        auto dst_ptr  = reinterpret_cast<TYPE*>(arg->dst_addr);

        uint32_t gid = blockIdx.x * blockDim.x + threadIdx.x;
        TYPE value = src0_ptr[gid];
        dst_ptr[gid] = (value < 0) ? TYPE(0) : value;
}
C ommon.h:
#ifndef TYPE
#define TYPE float
#endif

typedef struct {
  uint32_t num_points;
  uint64_t src0_addr;
  uint64_t dst_addr;
} kernel_arg_t;
```

Host Code: Setup

```
int main(void)
{
    const int n = 1 << 20;          // 1 M elements
    size_t bytes = n * sizeof(float);

    float *h_a = (float *) malloc(bytes);
    float *h_b = (float *) malloc(bytes);
    float *h_c = (float *) malloc(bytes);
    /* ... fill h_a and h_b ... */

    float *d_a, *d_b, *d_c;
    cudaMalloc(&d_a, bytes);
    cudaMalloc(&d_b, bytes);
    cudaMalloc(&d_c, bytes);

    cudaMemcpy(d_a, h_a, bytes, cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, h_b, bytes, cudaMemcpyHostToDevice);
```

Host Code: Launch and Clean Up

```
    int threads = 256;
    int blocks  = (n + threads - 1) / threads;

    vec_add<<<blocks, threads>>>(d_a, d_b, d_c, n);

    cudaMemcpy(h_c, d_c, bytes, cudaMemcpyDeviceToHost);

    cudaFree(d_a);  cudaFree(d_b);  cudaFree(d_c);
    free(h_a);  free(h_b);  free(h_c);
    return 0;
}
```

Why if (i < n) ?

Blocks come in whole units of 256 threads

n = 1000:

blocks = (1000 + 255) / 256 = 4

threads = 4 × 256 = **1024**

Threads 1000 .. 1023 have no element

without the check they write past the array

The <<< >>> Launch

```
kernel<<<blocks, threads>>>(args);
```

threads per block: a multiple of 32, at most 1024

128 or 256 is a good default

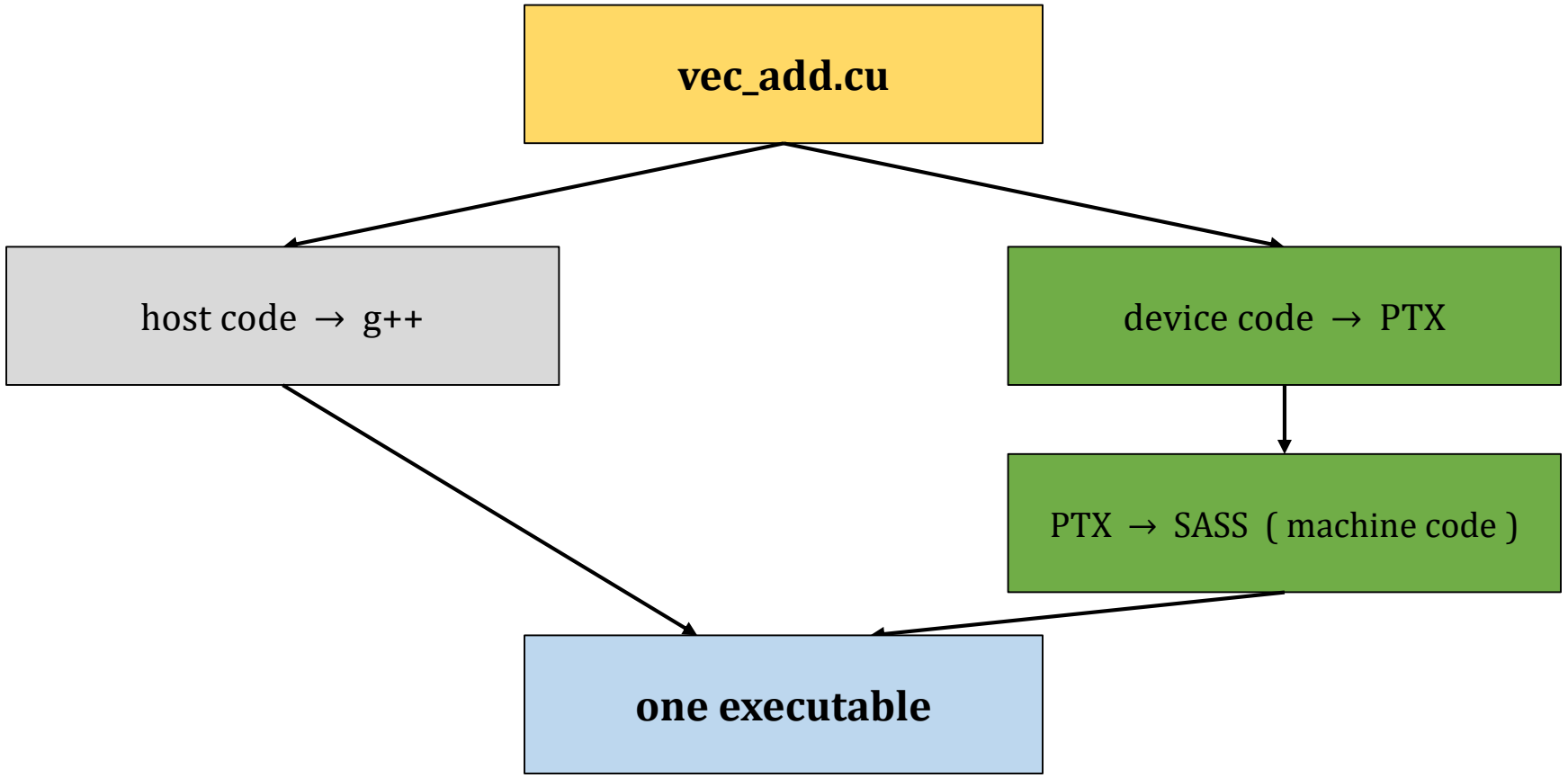
blocks: enough to cover the data

many more blocks than SMs is fine - and good

The launch **returns immediately**

Compiling: nvcc

```
nvcc -O2 -o vec_add vec_add.cu
```



Always Check for Errors

```
#define CUDA_CHECK(call)                                    \
do {                                                        \
    cudaError_t err = (call);                               \
    if (err != cudaSuccess) {                               \
        fprintf(stderr, "%s:%d  %s\n", __FILE__,            \
                __LINE__, cudaGetErrorString(err));         \
        exit(EXIT_FAILURE);                                 \
    }                                                       \
} while (0)

CUDA_CHECK(cudaMalloc(&d_a, bytes));
vec_add<<<blocks, threads>>>(d_a, d_b, d_c, n);
CUDA_CHECK(cudaGetLastError());     // bad launch?
CUDA_CHECK(cudaDeviceSynchronize());    // crashed?
```

Always Check for Errors

```
#define CUDA_CHECK(call_xz)    \   Add suffix to prevent conflicts
do {                                                    \
    cudaError_t err = (call_xz);                        \
    if (err != cudaSuccess) {                           \
        fprintf(stderr, "%s:%d  %s\n", __FILE__,        \
                __LINE__, cudaGetErrorString(err));     \
        exit(EXIT_FAILURE);                             \
    }                                                   \
} while (0)

CUDA_CHECK(cudaMalloc(&d_a, bytes));
vec_add<<<blocks, threads>>>(d_a, d_b, d_c, n);
CUDA_CHECK(cudaGetLastError());     // bad launch?
CUDA_CHECK(cudaDeviceSynchronize());    // crashed?
```

Unified Memory

```
float *a, *b, *c;
cudaMallocManaged(&a, bytes);
cudaMallocManaged(&b, bytes);
cudaMallocManaged(&c, bytes);

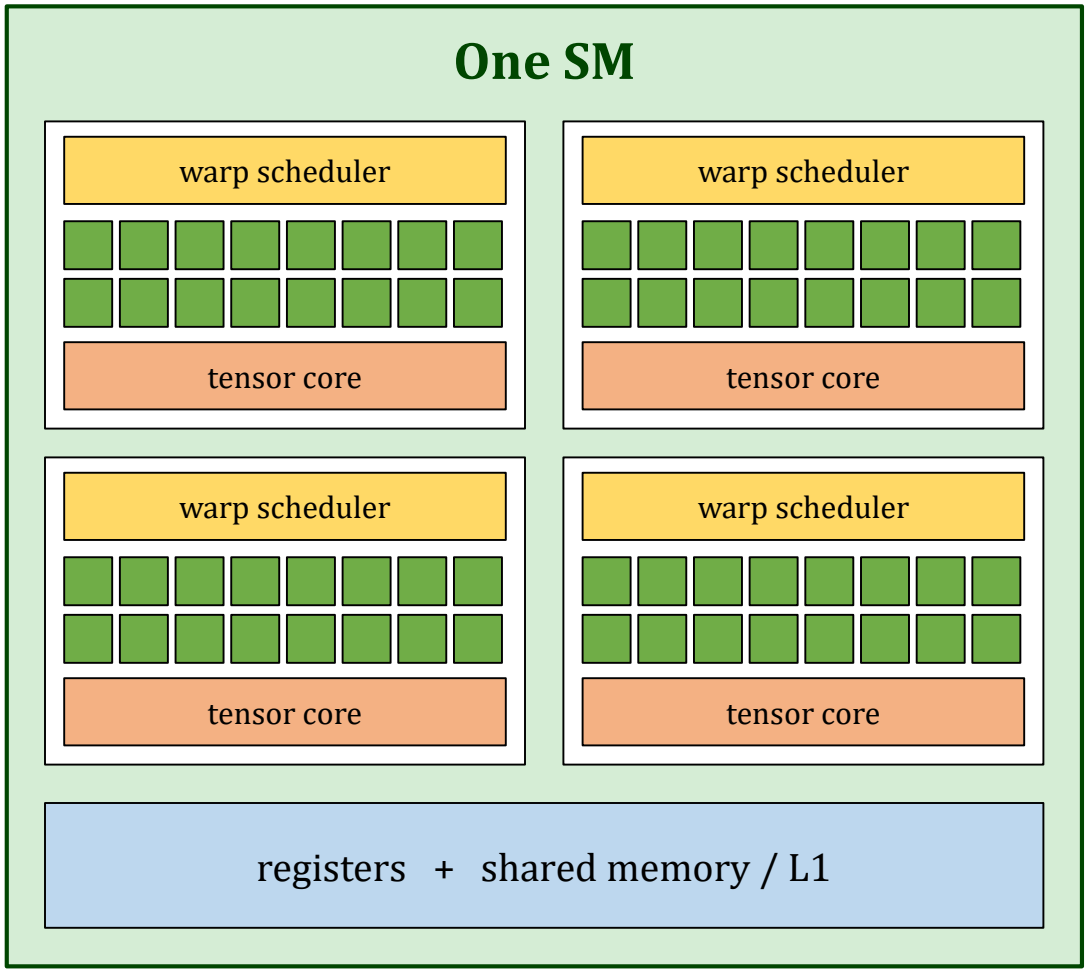
for (int i = 0; i < n; i++) {    // CPU writes
    a[i] = 1.0f;
    b[i] = 2.0f;
}
vec_add<<<blocks, threads>>>(a, b, c, n);
cudaDeviceSynchronize();         // wait for GPU
printf("%f\n", c[0]);            // CPU reads
```

One pointer works on both sides; pages move automatically

How the Hardware Runs It

streaming multiprocessors and warps

The Streaming Multiprocessor



H100 has **132** SMs

Each SM:

- 128 FP32 lanes
- 4 tensor cores
- up to 2048 threads
- 256 KB registers

Blocks Go to SMs

The hardware hands blocks to SMs as they free up

Small GPU: few SMs, blocks run in more rounds

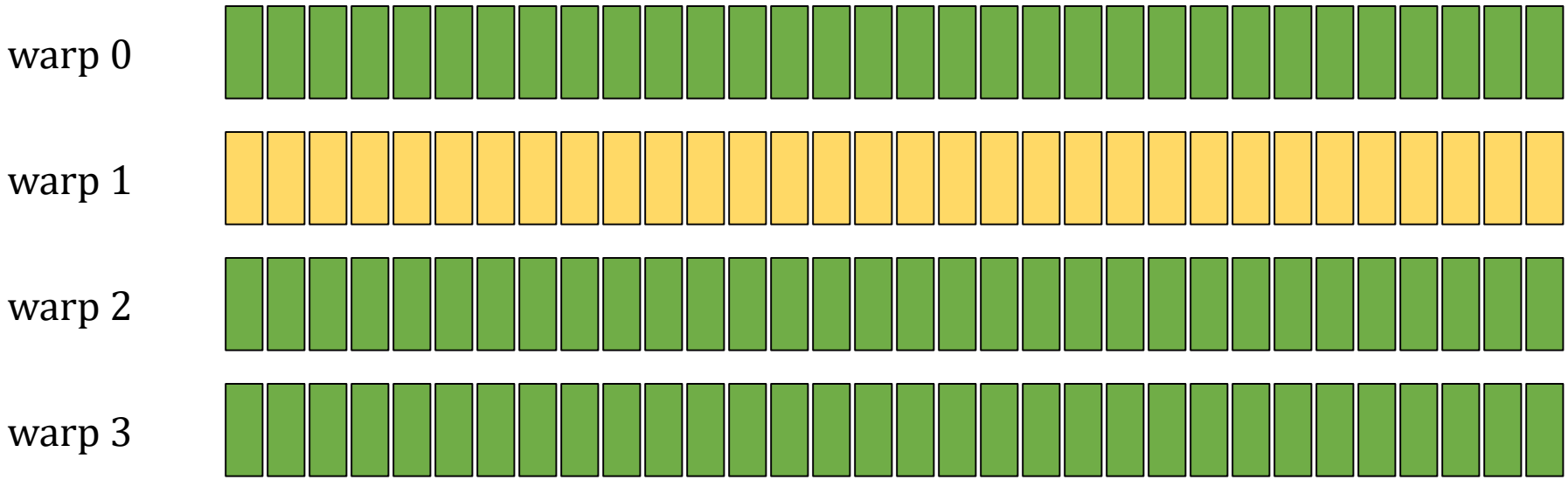
Big GPU: many SMs, blocks run in fewer rounds

Same program, no recompiling

This is why blocks must be independent

Warps: 32 Threads in Lockstep

A block of 128 threads = 4 warps



amber: warp 1 = threads 32 .. 63, one instruction for all

The SM issues **one instruction per warp**

All 32 threads execute it together: this is **SIMT**

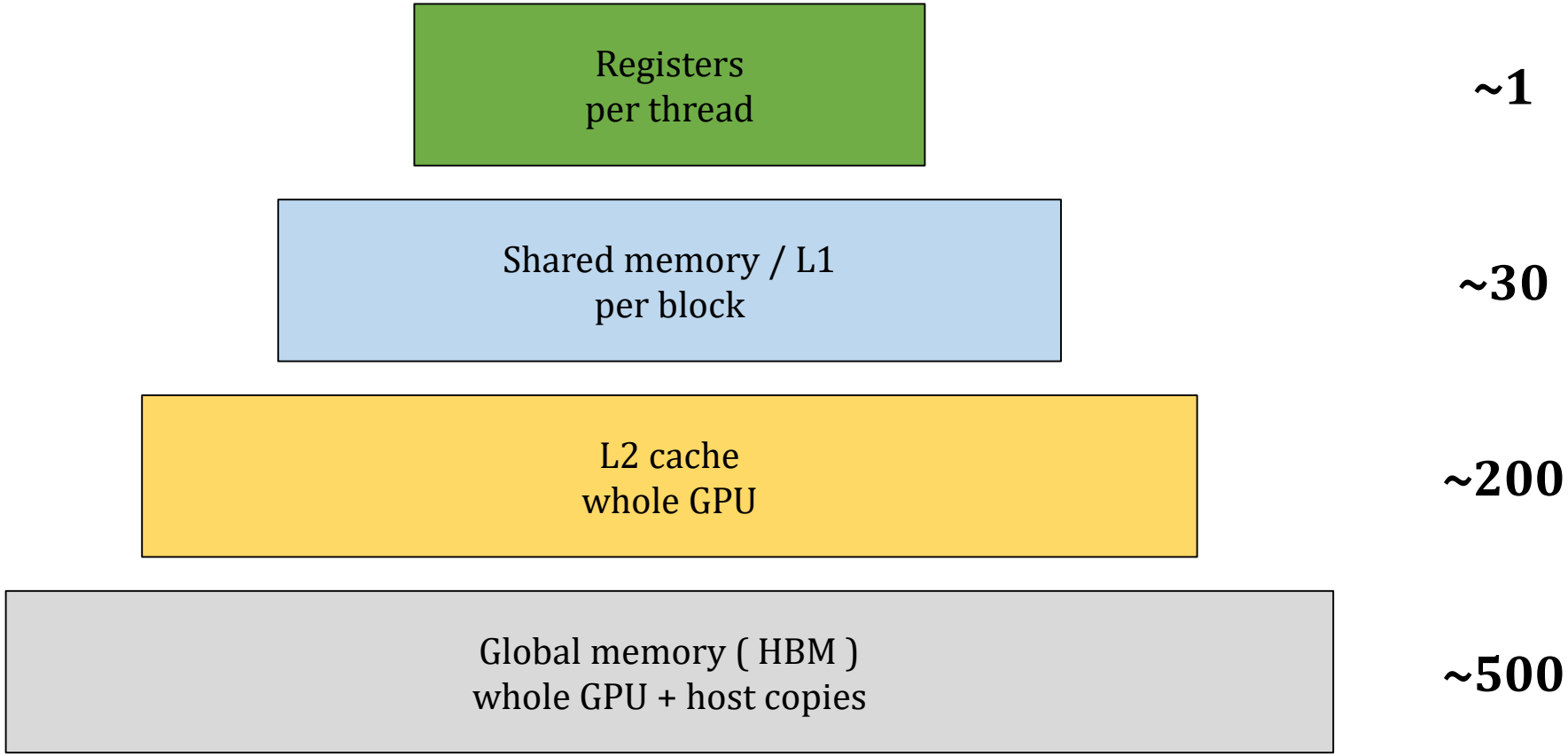
Branch Divergence

```
int i = blockIdx.x * blockDim.x + threadIdx.x;

if (threadIdx.x % 2 == 0)
    y[i] = a[i] * 2.0f;     // even threads
else
    y[i] = a[i] + 1.0f;     // odd threads
```

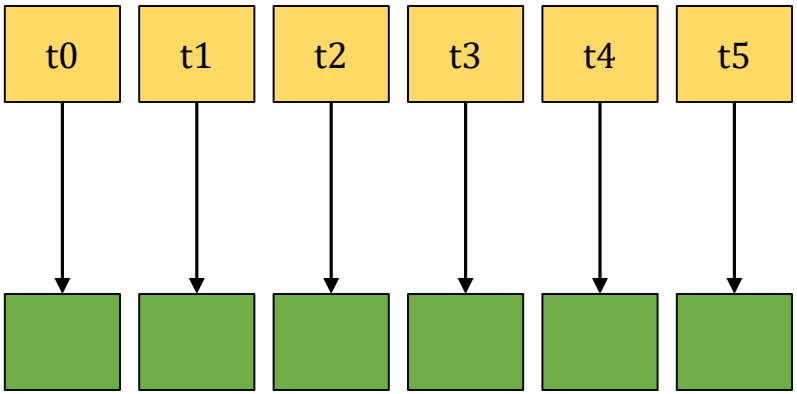
One warp runs both paths, with half its lanes switched off each time

The Memory Hierarchy



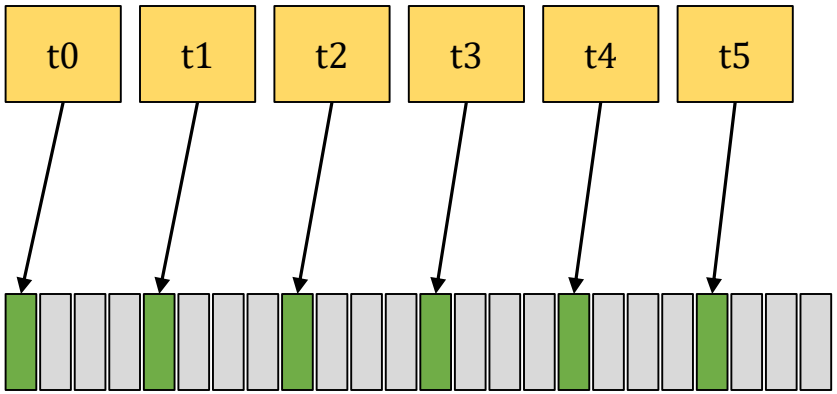
Coalesced Memory Access

Good: c[i]



one wide memory transaction

Bad: c[i * 8]



many transactions, most bytes wasted

Neighbouring threads should touch **neighbouring addresses**

The hardware merges a warp's 32 loads into a few wide ones

Occupancy

Occupancy = warps resident / maximum warps

More resident warps → more to switch to

→ better latency hiding

Limited by what each thread and block uses:

registers per thread

shared memory per block

Matrix Multiply

the operation that runs modern AI

C = A × B

Each element of C is a **dot product**:

$$C[r][c] = \Sigma\ A[r][k] \cdot B[k][c]$$

N × N matrices:

N^2 outputs, each needs N multiply-adds

total $\mathbf{2\ N^3}$ floating point operations

N = 4096: about 137 billion operations

Matrix Multiply on the CPU

```
void matmul(const float *A, const float *B,
                float *C, int N)
{
    for (int r = 0; r < N; r++)
        for (int c = 0; c < N; c++) {
            float sum = 0.0f;
            for (int k = 0; k < N; k++)
                sum += A[r * N + k] * B[k * N + c];
            C[r * N + c] = sum;
        }
}
```

Naive Matrix Multiply Kernel

```
__global__
void matmul(const float *A, const float *B,
                float *C, int N)
{
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < N && col < N) {
        float sum = 0.0f;
        for (int k = 0; k < N; k++)
            sum += A[row * N + k] * B[k * N + col];
        C[row * N + col] = sum;
    }
}
```

A 2-D Launch

```
dim3 threads(16, 16);          // 256 threads per block
dim3 blocks((N + 15) / 16,
            (N + 15) / 16);

matmul<<<blocks, threads>>>(d_A, d_B, d_C, N);
```

Each block computes a 16 × 16 tile of C

What Is Wrong with the Naive Kernel?

Each thread reads a whole row of A

and a whole column of B from **global memory**

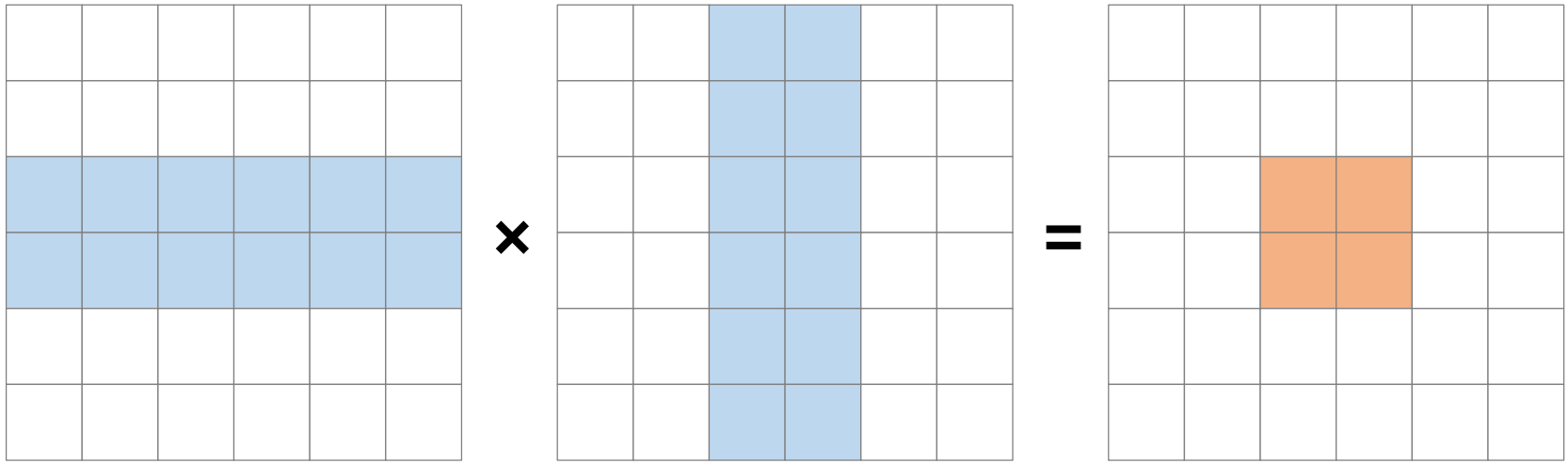
Every element of A is read **N times**

2 loads for every multiply-add:

the ALUs sit waiting on memory

Fix: **reuse** data from fast on-chip memory

Tiling



A **B** **C**

A block loads one **tile** of A and one of B into **shared memory**

All its threads reuse those tiles, then move to the next pair

Tiled Kernel: Setup

```
#define TILE 16

__global__
void matmul_tiled(const float *A, const float *B,
                  float *C, int N)
{
    __shared__ float As[TILE][TILE];    // one tile of A
    __shared__ float Bs[TILE][TILE];    // one tile of B

    int tx  = threadIdx.x,  ty = threadIdx.y;
    int row = blockIdx.y * TILE + ty;
    int col = blockIdx.x * TILE + tx;
    float sum = 0.0f;
```

Tiled Kernel: The Loop

```
    for (int t = 0; t < (N + TILE - 1) / TILE; t++) {
        int ac = t * TILE + tx;     // column in A
        int br = t * TILE + ty;     // row in B

        As[ty][tx] = (row < N && ac < N) ? A[row * N + ac] : 0.0f;
        Bs[ty][tx] = (br < N && col < N) ? B[br * N + col] : 0.0f;
        __syncthreads();

        for (int k = 0; k < TILE; k++)
            sum += As[ty][k] * Bs[k][tx];
        __syncthreads();
    }
    if (row < N && col < N)
        C[row * N + col] = sum;
}
```

Why __syncthreads() Twice?

A barrier: no thread passes until **all** in the block arrive

First: do not compute until the tiles are fully loaded

otherwise you read stale shared memory

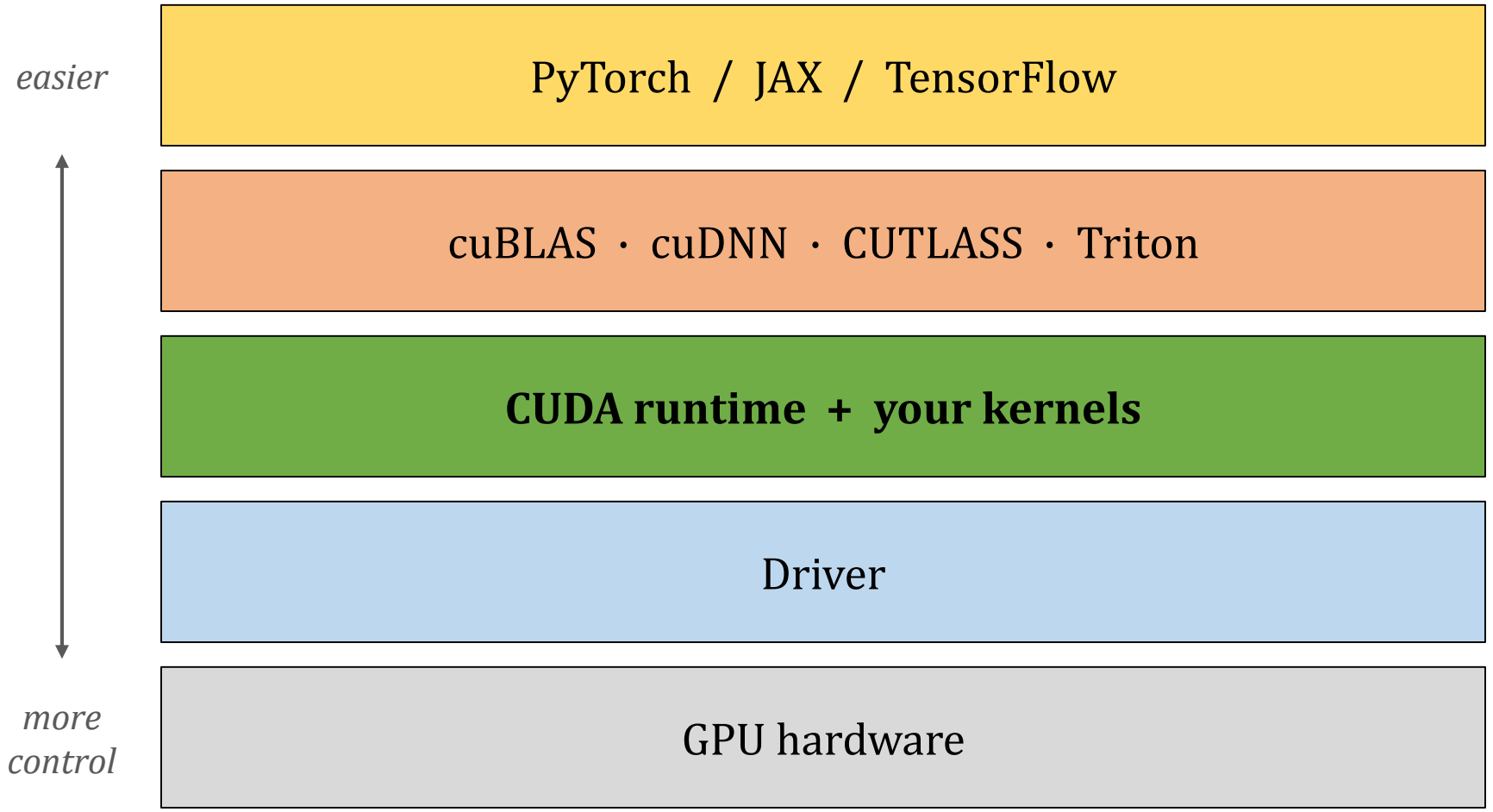
Second: do not load the next tile

until everyone has finished with this one

Above CUDA

libraries and frameworks

The Software Stack



cuBLAS

cuBLAS (CUDA Basic Linear Algebra Subroutines)

- NVIDIA GPU-accelerated library
- Implements standard basic linear algebra subprograms
- Used for high-performance numerical computing / deep learning

cuDNN

cuDNN (CUDA Deep Neural Network library)

- GPU-accelerated library from NVIDIA
- Provides highly optimized routines for deep learning tasks
- Provides building blocks (primitives) for deep neural networks
- Convolution, pooling, normalization, and attention layers.

CUTLASS

CUTLASS (CUDA Templates for Linear Algebra Subroutines and Solvers)

Open-source collection of CUDA C++ template abstractions and Python domain-specific languages (DSLs)

Designed to implement high-performance matrix-matrix multiplication (GEMM) and linear algebra computations on NVIDIA GPUs

Hierarchical Design: Decomposes complex linear algebra operations into thread-block tiles, warps, and individual threads matching the CUDA hardware model

pCuTe Library: Uses a core foundation called CuTe for defining multidimensional tensor layouts, memory spaces, and thread organization, simplifying complex data indexing

Mixed-Precision Support: Supports FP64, FP32, TF32, FP16, BF16, 8-bit/4-bit integers, and newer block-scaled formats like NVFP4 and MXFP4/MXFP8

4-bit floating point (NVFP4)

Bit Layout: Uses an E2M1 configuration consisting of 1 sign bit, 2 exponent bits, and 1 mantissa bit, covering a value range of approximately -6 to 6.

Two-Level Scaling: Employs fine-grained hierarchical scaling—using an FP8 scale for local micro-blocks of 16 values and a second FP32 scale for the entire tensor—to preserve dynamic range and global distribution.

Hardware Acceleration: Designed for native execution on NVIDIA Blackwell fifth-generation Tensor Cores, allowing direct 4-bit math processing with higher-precision accumulation.

4-bit floating point (MXFP4)

Bit Layout: Each weight uses only 4 bits in an E2M1 layout (1 sign bit, 2 exponent bits, and 1 mantissa bit).

Block sharing: Instead of scaling every number alone, it groups 32 values into a single block

Shared scale: The whole block shares one 8-bit exponent scaling factor. This method gives small groups a wide dynamic range without wasting memory.

8-bit floating point (MXFP8)

Bit Layout: Values are encoded using the FP8 E4M3 format (1 sign bit, 4 exponent bits, and 3 mantissa bits).
Block Scaling: Unlike standard per-tensor FP8, MXFP8 divides tensors into contiguous blocks of 32 elements along an axis.
Shared Scaling Factors: Each 32-element block shares a single unsigned exponent-only scaling factor in the E8M0 format (8 exponent bits, 0 mantissa bits).
Granularity Benefits: Fine-grained 1x32 scaling limits the impact of data outliers to their specific local block, preventing the severe quantization noise or clipping seen in traditional 8-bit formats
Claims near-lossless 16-bit (BF16) accuracy

OpenAI Triton

- Open-source, Python-embedded domain specific language (DSL) and compiler designed to write highly efficient custom GPU kernels.
- Triton abstracts the grid of individual hardware threads into a blocked program representation—meaning operations are executed on blocks of data rather than individual scalar elements.
- Operation Fusion: In standard deep learning setups, executing multiple individual functions sequentially forces data to move back and forth between the GPU's fast cache (shared memory) and its slower High Bandwidth Memory (HBM/VRAM). Triton fuses these operations into a single, unified kernel. Temporary data stays directly inside the Streaming Multiprocessor's (SM) shared memory, eliminating massive data-transfer overheads.
- Automatic Hardware Optimization: The Triton compiler takes the high-level Python Abstract Syntax Tree (AST) and automatically handles low-level optimizations like memory coalescing, shared memory management, software pipelining, and scheduling within SMs.
- Used by PyTorch 2.x

PyTorch

- Open-source machine learning and deep learning framework
- Provides a multi-dimensional array object called a Tensor that runs on GPUs and TPUs
- Autograd engine automatically calculates gradients (derivatives) required for training neural networks
- Uses a dynamic graph execution style so code runs line-by-line just like standard Python
- PyTorch 2.x and TorchInductor converts Python to highly optimized OpenAI Triton kernels

JAX

- Open-source, high-performance low level Python library developed by Google
- Designed to mimic Numerical Python (NumPy)
- Adds transformations that make it incredibly fast on hardware accelerators like GPUs and TPUs.
- Functional programming
- Treats your code as pure mathematical functions
- Low level

TensorFlow

- Open-source, end-to-end machine learning platform developed by Google
- Released in 2015
- Python bindings
- Historically dominant framework for building and training deep neural networks
- TensorFlow originally relied entirely on static computation graphs
- Now uses Eager execution which is dynamic but core strength remains static mapping

Calling cuBLAS

```
#include <cublas_v2.h>

cublasHandle_t handle;
cublasCreate(&handle);

const float alpha = 1.0f, beta = 0.0f;
// cuBLAS is column-major:  ask for  C^T = B^T A^T
cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N,
            N, N, N, &alpha,
            d_B, N,  d_A, N,
            &beta, d_C, N);

cublasDestroy(handle);
```

PyTorch: One Line

```
import torch

a = torch.randn(4096, 4096, device="cuda")
b = torch.randn(4096, 4096, device="cuda")

c = a @ b           # runs a cuBLAS kernel on the GPU

torch.cuda.synchronize()    # wait, like cudaDeviceSynchronize
print(c.shape)
```

Same five steps as before, hidden inside the library

Writing Kernels in Python: Triton

```
import triton
import triton.language as tl

@triton.jit
def add_kernel(x_ptr, y_ptr, out_ptr, n, BLOCK: tl.constexpr):
    pid  = tl.program_id(0)              # like blockIdx.x
    offs = pid * BLOCK + tl.arange(0, BLOCK)
    mask = offs < n                      # like if (i < n)
    x = tl.load(x_ptr + offs, mask=mask)
    y = tl.load(y_ptr + offs, mask=mask)
    tl.store(out_ptr + offs, x + y, mask=mask)

add_kernel[(triton.cdiv(n, 1024),)](x, y, out, n, BLOCK=1024)
```

Other Ways In

CuPy, **Numba**: NumPy-style arrays on the GPU

HIP: AMD's CUDA look-alike

OpenCL, **SYCL**: vendor-neutral standards

The model underneath is always the same:

grid → blocks → threads → warps

Key Ideas from Part 2

Write the kernel for **one** thread

Index with blockIdx, blockDim, threadIdx

Warps of 32 run in lockstep: avoid divergence

Neighbours touch neighbours: coalesce

Reuse data in **shared memory**

In practice: call cuBLAS or PyTorch

Let's Write Some Kernels

Exercise: Three Matrix Multiplies

1. Build vec_add and check every element against the CPU
2. Write the naive and tiled matmul kernels
3. Time each for N = 512, 1024, 2048 using cudaEvent
4. Time cublasSgemm and PyTorch a @ b at the same sizes
5. Report GFLOP/s = $2\,N^3$ / time. Plot all four
6. Try TILE = 8, 16, 32. Explain what you see